

NotebookLM Source: Troubleshooting e Guia de Erros Comuns (SQLite + Event Log)

Esta guia lista os erros mais recorrentes durante o desenvolvimento e integração da arquitetura SQLite puro da Semana 15 e como corrigi-los rapidamente.

1. Erro de Acesso por Nome de Coluna

O Erro:

```
TypeError: tuple indices must be integers or slices, not str
```

- **Causa:** Você tentou acessar uma coluna do banco pelo nome (ex: `row["ref"]` ou `row["sku"]` no repositório), mas a conexão SQLite3 não foi configurada para retornar os dados formatados como dicionários/objetos de linha.
- **Solução:** Garanta que a propriedade `row_factory` seja definida como `sqlite3.Row` imediatamente após abrir a conexão em `database.py`:

```
def get_connection():  
    conn = sqlite3.connect("estoque.db")  
    conn.row_factory = sqlite3.Row # <- ESSENCIAL  
    return conn
```

2. Erro de Importação Circular (Circular Imports)

O Erro:

```
ImportError: cannot import name '...' from partially initialized module '...' (most likely due to a circular import)
```

- **Causa:** `domain.py` importa `messages.py` (para usar eventos como `Allocated` ou `OutOfStock`) e `messages.py` importa algo do domínio, criando um ciclo de dependência.

- **Solução (O Truque do Professor):**

Para manter o domínio limpo e evitar importações circulares, **importar os eventos dinamicamente dentro dos métodos**, em vez de importá-los no topo do arquivo `domain.py`:

```
# domain.py  
class Product:  
    def allocate(self, line):  
        # IMPORT INTERNO/DINÂMICO PARA EVITAR CICLO  
        from messages import Allocated, OutOfStock  
  
        # ... resto do código  
        self.events.append(Allocated(orderid=line.orderid, ...))
```

3. Tabela não encontrada nos Testes (OperationalError)

O Erro:

```
sqlite3.OperationalError: no such table: batches (ou allocations / event_log)
```

- **Causa:** O banco de dados de testes foi iniciado, mas as tabelas ainda não foram geradas no arquivo **.db** correspondente, ou o script de testes rodou a partir de um diretório diferente, criando um arquivo **.db** vazio no lugar errado.

- **Solução:**

1. Certifique-se de chamar a função **create_tables()** no início do seu arquivo **app.py**.
2. No arquivo **conftest.py** dos testes, adicione a chamada de criação de tabelas para garantir que o banco temporário de testes seja inicializado corretamente:

```
# conftest.py
from database import create_tables
create_tables() # Gera as tabelas físicas antes de rodar os testes
```

4. Banco de Dados Bloqueado (Database is Locked)

O Erro:

```
sqlite3.OperationalError: database is locked
```

- **Causa:** O SQLite3 é um banco de arquivo único de escrita exclusiva. Se você abrir uma conexão e não fechá-la (**conn.close()**), qualquer outra escrita concorrente (ou outro teste rodando) ficará travada esperando a liberação do arquivo.

- **Solução:**

1. Certifique-se de que o UOW fecha a conexão no método **__exit__**:

```
def __exit__(self, exc_type, exc_value, traceback):
    if exc_type:
        self.conn.rollback()
    self.conn.close() # <- NUNCA ESQUECER
```

2. Se você criar conexões auxiliares em rotas do **app.py** (como no **GET /batches**), feche a conexão no bloco **finally** ou use **conn.close()** antes de dar o **return**.

5. Eventos Duplicados ou Perdidos no Log

Sintoma:

O consumidor de eventos exibe mensagens duplicadas ou deixa de mostrar eventos novos.

- **Causa:** O ponteiro **last_id** no loop do **event_consumer.py** foi reiniciado ou não está sendo atualizado corretamente a cada iteração de eventos lidos do banco.

- **Solução:** Certifique-se de atualizar **last_id** para o valor máximo da coluna **id** retornada na consulta de log:

```
for row in rows:
    # Processa o evento...
    last_id = row["id"] # <- ATUALIZA O PONTEIRO
```

6. Erro de Tipo ao Enviar Payload no Log

O Erro:

```
TypeError: Object of type Allocated is not JSON serializable
```

- **Causa:** O módulo `json.dumps()` tentou serializar o objeto de classe do evento diretamente em string JSON.
- **Solução:** Passe o dicionário de atributos do objeto (`event.__dict__`) na serialização:

```
# publisher.py
def publish_event(channel, event):
    payload = json.dumps(event.__dict__) # <- CONVERTE PARA DICIONÁRIO
    # ...
```